

Kontrollstrukturen in Python

if, Schleifen, break, pass, try-except, bedingte Ausdrücke und match-case, jeweils im direkten Vergleich mit Java

Kontrollstrukturen steuern, in welcher Reihenfolge und unter welchen Bedingungen Code ausgeführt wird. Man unterscheidet grob zwei Arten: **Fallunterscheidungen** (ein Block läuft nur, wenn eine Bedingung erfüllt ist) und **Schleifen** (ein Block läuft mehrmals). Dazu kommen Hilfsmittel wie **break**, **pass** und die Fehlerbehandlung mit **try**. Python und Java können im Grunde dasselbe, schreiben es aber unterschiedlich. Der auffälligste Unterschied: Python erkennt Blöcke an der **Einrückung** und einem Doppelpunkt, Java an **geschweiften Klammern** und Semikolons.

1. Die Kontrollstrukturen einzeln

if / elif / else

Der Block unter **if** wird nur ausgeführt, wenn die Bedingung wahr ist. Weitere Bedingungen werden nur geprüft, wenn alle vorherigen falsch waren, und **else** fängt den Rest ab. Python schreibt **elif**, Java **else if**. In Java muss die Bedingung außerdem in runden Klammern stehen.

Python	Java
<pre>punkte = 78 if punkte >= 90: print("Sehr gut") elif punkte >= 50: print("Bestanden") else: print("Nicht bestanden")</pre>	<pre>int punkte = 78; if (punkte >= 90) { System.out.println("Sehr gut"); } else if (punkte >= 50) { System.out.println("Bestanden"); } else { System.out.println("Nicht bestanden"); }</pre>

for-Schleife

Die Python-**for**-Schleife läuft immer über eine Sequenz (Liste, String, **range()** usw.). Java hat die klassische Zählschleife mit Start, Bedingung und Schritt sowie die for-each-Schleife, die der Python-Variante entspricht.

Python	Java
<pre>for i in range(1, 4): print(i) for name in ["Anna", "Ben"]: print(name)</pre>	<pre>for (int i = 1; i < 4; i++) { System.out.println(i); } String[] namen = {"Anna", "Ben"}; for (String name : namen) { System.out.println(name); }</pre>

while-Schleife

Die **while**-Schleife läuft, solange die Bedingung wahr ist, und funktioniert in beiden Sprachen gleich. Python kennt allerdings kein **++** und keine **do-while**-Schleife, die in Java mindestens einmal durchläuft.

Python	Java
<pre>n = 3 while n > 0: print(n) n -= 1 # kein n-- in Python</pre>	<pre>int n = 3; while (n > 0) { System.out.println(n); n--; } // zusätzlich: do { ... } while (n > 0);</pre>

break (und continue)

break beendet die Schleife sofort, **continue** springt zum nächsten Durchlauf. Beides verhält sich in Python und Java gleich. Java kann zusätzlich mit einem **Label** (**break aussen;**) gleich mehrere verschachtelte Schleifen verlassen, das gibt es in Python nicht.

Python	Java
<pre>for z in [4, 7, 12, 3]: if z > 10: print("Gefunden:", z) break</pre>	<pre>for (int z : new int[]{4, 7, 12, 3}) { if (z > 10) { System.out.println("Gefunden: " + z); break; } }</pre>

pass

Python verlangt in jedem Block mindestens eine Anweisung, sonst gibt es einen Syntaxfehler.

pass ist eine leere Anweisung, die genau diese Lücke füllt, typischerweise als Platzhalter für Code, der später kommt. Java braucht dafür kein eigenes Schlüsselwort, weil ein leerer Block **{ }** einfach erlaubt ist.

Python	Java
<pre>def spaeter(): pass if x > 0: pass # TODO</pre>	<pre>void spaeter() { } if (x > 0) { // TODO }</pre>

try / except (Fehlerbehandlung)

Code, der einen Fehler auslösen kann, kommt in den **try**-Block. Tritt der Fehler auf, springt das Programm in den passenden Behandlungsblock, statt abzustürzen. Python nennt ihn **except**, Java **catch**. Nur Python hat zusätzlich einen **else**-Zweig, der läuft, wenn *kein* Fehler aufgetreten ist. **finally** läuft in beiden Sprachen immer.

Python	Java
<pre>try: zahl = int("abc") except ValueError: print("Keine Zahl!") else: print("Alles ok") finally: print("Fertig")</pre>	<pre>try { int zahl = Integer.parseInt("abc"); } catch (NumberFormatException e) { System.out.println("Keine Zahl!"); } finally { System.out.println("Fertig"); } // kein else-Zweig in Java</pre>

Bedingte Ausdrücke

Ein bedingter Ausdruck liefert je nach Bedingung einen von zwei Werten und passt damit eine einfache if/else-Zuweisung in eine Zeile. Python liest sich wie Englisch: **A if Bedingung else B**. Java verwendet den ternären Operator **Bedingung ? A : B**. In beiden Fällen wird zuerst die Bedingung ausgewertet und danach nur einer der beiden Werte.

Python	Java
<pre>alter = 17 typ = "Erwachsen" if alter >= 18 else "Kind" print(typ) # Kind</pre>	<pre>int alter = 17; String typ = (alter >= 18) ? "Erwachsen" : "Kind"; System.out.println(typ); // Kind</pre>

Switch-Case: match-case gegen switch

Seit Python 3.10 gibt es **match/case**. Mehrere Werte verbindet man mit **|**, der Default-Fall heißt **case _**. Nach dem ersten Treffer ist Schluss, ein **break** ist nicht nötig. Beim klassischen Java-**switch** ist das anders: Fehlt das **break**, läuft das Programm einfach in den nächsten **case** weiter (**Fallthrough**).

Python	Java
<pre>befehl = "stopp" match befehl: case "start": print("Startet") case "stopp" "halt": print("Stoppt") case _: print("Unbekannt")</pre>	<pre>String befehl = "stopp"; switch (befehl) { case "start": System.out.println("Startet"); break; case "stopp": case "halt": System.out.println("Stoppt"); break; default: System.out.println("Unbekannt"); }</pre>

Seit Java 14 gibt es zusätzlich die kürzere Pfeil-Schreibweise ohne Fallthrough, die dem Python-Stil schon recht nahekommt:

```
switch (befehl) {
    case "start"      -> System.out.println("Startet");
    case "stopp", "halt" -> System.out.println("Stoppt");
    default          -> System.out.println("Unbekannt");
}
```

Python-`match` kann außerdem mehr als nur Werte vergleichen: Es kann Datenstrukturen zerlegen und mit einem sogenannten Guard eine Zusatzbedingung prüfen, etwa `case (x, y) if x == y:`. Java hat ab Version 21 ebenfalls Pattern Matching im `switch`, dort heißt die Zusatzbedingung `when`.

2. Alles in einem Beispiel: Das Ratespiel in Python

Das kleine Spiel kombiniert alle Kontrollstrukturen: Der Computer denkt sich eine Zahl zwischen 1 und 10 aus, du rätst. Die `while True`-Schleife hält das Spiel am Laufen, bis ein `break` kommt. `match` prüft zuerst, ob ein Befehl eingegeben wurde: Bei „ende“ beendet `break` die Schleife, bei „hilfe“ verrät ein bedingter Ausdruck, ob die Zahl gerade oder ungerade ist, und `continue` startet die nächste Runde. Sonst macht `pass` nichts. Danach versucht `try`, die Eingabe in eine Zahl umzuwandeln, und `except ValueError` fängt Buchstaben ab. `if/elif/else` vergleicht, und beim Treffer gibt eine `for`-Schleife drei Sternchen aus. Die Kommentare in Großbuchstaben markieren jede Struktur.

```
import random

geheim = random.randint(1, 10)
versuche = 0

print("Rate eine Zahl zwischen 1 und 10!")
print("Befehle: 'hilfe' = Tipp bekommen, 'ende' = aufhören")

while True:                                # SCHLEIFE (while): läuft, bis break kommt
    eingabe = input("Dein Tipp: ").strip().lower()

    match eingabe:                          # MATCH-CASE (Pythons switch-case)
        case "ende":
            print("Spiel abgebrochen. Die Zahl war", geheim)
            break                          # BREAK: Schleife sofort verlassen
        case "hilfe":
            # BEDINGTER AUSDRUCK: A if Bedingung else B
            print("Tipp: Die Zahl ist", "gerade" if geheim % 2 == 0 else "ungerade")
            continue                      # nächste Runde
        case _:
            pass                          # PASS: nichts tun, einfach unten weitermachen

    try:                                    # TRY-EXCEPT: Fehler abfangen
        tipp = int(eingabe)
    except ValueError:
        print("Das war keine Zahl, versuch's nochmal.")
        continue

    versuche += 1

    if tipp < geheim:                       # IF / ELIF / ELSE
        print("Zu klein!")
    elif tipp > geheim:
        print("Zu groß!")
    else:
        wort = "Versuch" if versuche == 1 else "Versuchen" # noch ein bedingter Ausdruck
        print(f"Richtig! Du hast es in {versuche} {wort} geschafft.")
        for i in range(3):                # SCHLEIFE (for): läuft genau 3-mal
            print("*", end=" ")
        print()
        break
```

Starten mit `python ratespiel.py` (Python 3.10 oder neuer, wegen `match`).

3. Dasselbe Ratespiel in Java

Hier ist exakt dasselbe Programm in Java. Beim direkten Vergleich sieht man schön, wie viel mehr „Drumherum“ Java braucht: eine Klasse, eine `main`-Methode, Typangaben bei jeder Variable, einen `Scanner` für die Eingabe sowie Klammern und Semikolons.

```
import java.util.Random;
import java.util.Scanner;

public class Ratespiel {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int geheim = new Random().nextInt(10) + 1;
        int versuche = 0;

        System.out.println("Rate eine Zahl zwischen 1 und 10!");
        System.out.println("Befehle: 'hilfe' = Tipp, 'ende' = aufhören");

        spiel:                                // Label für break
        while (true) {                        // SCHLEIFE (while)
            System.out.print("Dein Tipp: ");
            String eingabe = scanner.nextLine().trim().toLowerCase();

            switch (eingabe) {                // SWITCH-CASE
                case "ende":
                    System.out.println("Abgebrochen. Zahl war " + geheim);
                    break spiel;              // BREAK mit Label: verlässt while
                case "hilfe":
                    // BEDINGTER AUSDRUCK: Bedingung ? A : B
                    System.out.println("Tipp: Die Zahl ist "
                        + (geheim % 2 == 0 ? "gerade" : "ungerade"));
                    continue;
                default:
                    // nichts tun (wie pass in Python)
                    break;                    // verlässt NUR das switch
            }

            int tipp;
            try {                             // TRY-CATCH
                tipp = Integer.parseInt(eingabe);
            } catch (NumberFormatException e) {
                System.out.println("Das war keine Zahl, nochmal.");
                continue;
            }

            versuche++;

            if (tipp < geheim) {                // IF / ELSE IF / ELSE
                System.out.println("Zu klein!");
            } else if (tipp > geheim) {
                System.out.println("Zu groß!");
            } else {
                String wort = (versuche == 1) ? "Versuch" : "Versuchen";
                System.out.println("Richtig! Du hast es in "
                    + versuche + " " + wort + " geschafft.");
                for (int i = 0; i < 3; i++) {    // SCHLEIFE (for)
                    System.out.print("* ");
                }
                System.out.println();
                break;
            }
        }
        scanner.close();
    }
}
```

Die wichtigste Falle in diesem Beispiel

In Python beendet das `break` im `case "ende"` die `while`-Schleife, weil `match` selbst gar kein `break` kennt. In Java dagegen gehört ein `break` innerhalb eines `switch` zum `switch` und würde nur diesen verlassen. Das Spiel liefere dann einfach weiter. Deshalb bekommt die Schleife in Java das Label `spiel:`, und `break spiel;` verlässt gezielt die Schleife. `continue` funktioniert dagegen in beiden Sprachen gleich, weil es sich immer auf die Schleife bezieht.

Starten mit `java Ratespiel.java` (ab Java 11 geht das ohne vorheriges Kompilieren) oder klassisch mit `javac Ratespiel.java` und danach `java Ratespiel`.

4. Gegenüberstellung: Was ist in Python anders?

Thema	Python	Java
Codeblöcke	Einrückung und Doppelpunkt <code>;</code> ; die Einrückung ist Pflicht	Geschweifte Klammern <code>{ }</code> ; Einrückung nur für die Lesbarkeit
Anweisungsende	Zeilenende	Semikolon <code>;</code>
Bedingung	Keine Klammern nötig: <code>if x > 0:</code>	Klammern Pflicht: <code>if (x > 0)</code>
Sonst-wenn	<code>elif</code>	<code>else if</code>
Logische Operatoren	<code>and</code> , <code>or</code> , <code>not</code>	<code>&&</code> , <code> </code> , <code>!</code>
Variablen	Ohne Typangabe (dynamisch): <code>x = 5</code>	Mit Typ (statisch): <code>int x = 5;</code> , seit Java 10 auch <code>var</code>
Hochzählen	<code>x += 1</code> (kein <code>++</code>)	<code>x++</code> oder <code>x += 1</code>
for-Schleife	Nur über Sequenzen: <code>for x in liste:</code> , Zählen mit <code>range()</code>	Zählschleife <code>for (int i = 0; i < n; i++)</code> und <code>for-each</code> <code>for (T x : liste)</code>
do-while	Gibt es nicht	<code>do { ... } while (bed);</code> läuft mindestens einmal
else nach Schleife	<code>for/while ... else:</code> läuft, wenn kein <code>break</code> kam	Gibt es nicht
break mit Label	Nicht möglich	<code>break aussen;</code> verlässt eine äußere Schleife
Leerer Block	<code>pass</code> ist nötig	Leerer Block <code>{ }</code> genügt
Fehler abfangen	<code>try / except / else / finally</code>	<code>try / catch / finally</code> (kein <code>else</code>)
Fehler auslösen	<code>raise ValueError(...)</code>	<code>throw new IllegalArgumentException(...);</code>
Pflicht zur Behandlung	Keine: Fehler muss man nicht abfangen	Checked Exceptions müssen gefangen oder mit <code>throws</code> deklariert werden
Bedingter Ausdruck	<code>A if bed else B</code>	<code>bed ? A : B</code>
Mehrfachauswahl	<code>match / case</code> (ab Python 3.10)	<code>switch</code> (klassisch; ab Java 14 mit <code>-></code> ; ab 21 Pattern Matching)
Fallthrough	Nie: erster Treffer, dann Ende	Klassisch ja: ohne <code>break</code> läuft es in den nächsten <code>case</code>
Default-Fall	<code>case _:</code>	<code>default:</code>
Mehrere Werte	<code>case "a" "b":</code>	<code>case "a", "b" -></code> oder mehrere <code>case</code> -Zeilen untereinander

Thema	Python	Java
Guard / Zusatzbedingung	<code>case x if x > 10:</code>	<code>case Integer i when i > 10 -></code> (ab Java 21)
Strings vergleichen	<code>==</code> vergleicht den Inhalt	<code>.equals()</code> verwenden; <code>==</code> vergleicht nur Referenzen

5. Typische Stolperfallen beim Umstieg

Einrückung ist in Python Syntax. Was in Java nur schlecht aussieht, ist in Python ein Fehler (`IndentationError`) oder, schlimmer, ändert still die Logik, weil eine Zeile plötzlich zu einem anderen Block gehört. Am besten immer 4 Leerzeichen verwenden und Tabs und Leerzeichen nie mischen.

break im switch. Wer von Python kommt, vergisst in Java gerne das `break` am Ende eines `case` und wundert sich, dass mehrere Fälle ausgeführt werden. Umgekehrt verlässt ein `break` im Java-`switch` nicht die umgebende Schleife, wie im Ratespiel gezeigt. Die neue Pfeil-Schreibweise ab Java 14 vermeidet das erste Problem komplett.

Strings mit `==` vergleichen. In Python ist `name == "Anna"` völlig korrekt. In Java liefert `==` bei Strings oft ein falsches Ergebnis, weil es prüft, ob es dasselbe Objekt ist. Richtig ist `name.equals("Anna")`. Der Java-`switch` auf Strings macht das intern schon richtig.

Kein `++` in Python. `x++` ist in Python ein Syntaxfehler, und `++x` ist zwar erlaubt, tut aber nichts (zweimal plus davor). Immer `x += 1` schreiben.

match gibt es erst ab Python 3.10. Auf älteren Versionen gibt es einen Syntaxfehler. Die Version prüft man mit `python --version`. Als Ersatz dienen eine `if/elif/else`-Kette oder ein Dictionary, etwa `aktionen.get(befehl, "Unbekannt")`.

Quellen: Ernesti/Kaiser, *Python 3 – Das umfassende Handbuch*, Rheinwerk Openbook, Kapitel 5 (openbook.rheinwerk-verlag.de/python/05_001.html); DataCamp, *Python Switch Case Statement: A Beginner's Guide* (datacamp.com/tutorial/python-switch-case). Hinweis: Das Rheinwerk-Buch stammt aus der Zeit vor Python 3.10 und schreibt deshalb noch, dass es kein `switch/case` gibt.